
samna-dynapse1

Release 0.1

NCS-INI

May 02, 2022

CONTENTS

1	Contents	1
1.1	Getting started	1
1.2	Tutorial 1: basic usage	2
1.3	Tutorial 2: train a network on chip using STDP	9
1.4	Modules	11
1.5	Debugging	33
1.6	APIs summary	34
	Python Module Index	39
	Index	41

CONTENTS

1.1 Getting started

1.1.1 About DYNAP-SE1

A DYNAP-SE1 board has 4 chips and each chip contains 4 cores featuring 256 silicon neurons. We call the state of the 4 chips a configuration. Each silicon neuron has 64 synapses to receive the incoming connections of its presynaptic neurons. Each core holds a parameter group to store the neuronal and synaptic parameters for its 256 neurons and their incoming synapses. The neurons in the same core share the same parameter values. There are 4 types of synapses: AMPA, NMDA, GABA_A, and GABA_B. 1024 spike generators are available implemented on the FPGA. Each of the spike generator can be used as a Poisson spike generator or a normal FPGA spike generator.

1.1.2 How to use it?

To use DYNAP-SE1, we recommend [Samna](#) and its Python utilities [library](#). The latter is developed primarily by the [NCS group](#) of [INI](#). DYNAP-SE1 related section in Samna documentation is [here](#) which describes some basic core APIs. The Python repository adds more features to Samna for DYNAP-SE1, which wraps around the plain Samna classes and functions to provide more user-friendly and convenient Python APIs for constructing networks, configuring hardware parameters, monitoring neurons activity, implementing learning algorithms, etc.

To get started, choose a directory and do the following to download the library:

```
git clone git@gitlab.com:neuroinf/ctxctl_contrib.git
cd ctxctl_contrib
git checkout samna-dynapse1
```

1.1.3 Utilities library overview

The Python files (libraries) in the *samna-dynapse1* branch of the Python [repository](#) have the following functions:

- *dynapse1constants.py* defines the hardware specific parameters.
- *dynapse1utils.py* contains functions that wraps around low-level Samna APIs for DYNAP-SE1 users.
- *netgen.py* includes basic network elements, e.g. neuron groups, synapses and Winner-Take-All

(WTA) structures, that can be added into `NetworkGenerator` to implement Spiking Neural Networks (SNNs) on DYNAP-SE1 processor.

- The *example* folder has some example scripts to use the DYNAP-SE1 board. *example.py* gives a

simple example of how to implement a network on DYNAP-SE1 and *example_fpga_gen.py* changes the input from Poisson to constant spikes. *example_neuron_group.py* shows how to use building blocks in *netgen.py* to construct a network. Some example parameter files can be found in *example_parameter_files* folder. The other folders hold some testing scripts.

- *params.py* gives some board parameter configurations as examples.
- *stdp.py* and *stdp_utils.py* provide a learning framework for STDP-like algorithms.

Implementation of triplet-STDP is given as an example, and other algorithms can be implemented by users. An example of training the plastic connections between a presynaptic and a postsynaptic neuron group is in *example/test_stdp.py*.

1.2 Tutorial 1: basic usage

1.2.1 Import required libraries

Here, we import the related files as the following:

```
import samna.dynapse1 as dyn1
from dynapse1constants import *
import dynapse1utils as ut
import netgen as n
from netgen import Neuron
```

1.2.2 Open DYNAP-SE1 board to get Dynapse1Model

You can open the board with or without the Graphical User Interfaces (GUI) using *open_dynapse1*.

With the GUI, you should see blinking spikes of the firing silicon neurons.

```
model, gui_process = ut.open_dynapse1()
```

Without the GUI, if your PC cannot have the required visualization libraries e.g. glx.

```
model, gui_process = ut.open_dynapse1(gui=False)
```

`gui_process` will be empty if `gui=False`.

`model` is `Dynapse1Model`, with which you can configure DYNAP-SE1 board, get the current configuration of the board, set the parameters, set up the stimulus, get the spikes out of DYNAP-SE1 and so on.

1.2.3 Create a network

We need to use the `NetworkGenerator` to first create a network specifying the connections among neurons, then make a DYNAP-SE1 configuration out of this network, and apply the configuration to the board.

To define the network, you need to use `Neuron` instead of `Dynapse1Neuron` as the basic unit, and add connections between them to the network. The former only holds the topology-related information of the network, the latter maintains the hardware-specific data structures required by the `Dynapse1Configuration`.

Apart from the basic unit `Neuron`, we provide some building blocks, e.g. `NeuronGroup`, `Synapses`, `WTA_connections` that uses groups of `Neuron` to construct the network.

Define neuron groups

Create 2 neuron groups, one with spike generators and the other with neurons, using NeuronGroup

```
chip1 = 0
core1 = 0
nids1 = [101,102,103]

chip2 = 0
core2 = 1
nids2 = [14,15,16]

# create spikegen group by selecting the spikegens you want to use
pre_group = NeuronGroup(chip1,core1,nids1,is_spike_gen=True)
# optional prints
print('pre_group', pre_group.chip_id, pre_group.core_id, pre_group.neuron_ids,pre_group.
↳neurons)

# create neuron group by selecting the neurons you want to use
post_group = NeuronGroup(chip2,core2,nids2)
# optional prints
print('post_group',post_group.chip_id, post_group.core_id, post_group.neuron_ids,post_
↳group.neurons)
```

Define synapses between neuron groups

Define connections between the 2 neuron groups using Synapses. Connection pattern can be defined using conn_type, weight_matrix or pre_list and post_list. conn_type can be 'one2one' or 'all2all'. mux_conn can be used to duplicate the defined connectivity by multiplying the weights by mux_conn.

- One-to-one connections:

```
syn = Synapses(pre_group, post_group, dyn1.Dynapse1SynType.AMPA, conn_type='one2one')
```

- All-to-all random connections with possibility of 0.7. When 'all2all' is used, p can be used to specify the possibility of all2all random connections and rand_seed can be defined.

```
syn = Synapses(pre_group, post_group, dyn1.Dynapse1SynType.NMDA, conn_type='all2all',
↳p=0.7, rand_seed=60)
```

- Connection pattern defined by weight matrix:

```
int_w = np.array([
    [1, 0, 0],
    [1, 1, 0],
    [1, 1, 2],
])
syn = Synapses(pre_group, post_group, dyn1.Dynapse1SynType.AMPA, weight_matrix=int_w)
```

- Self-defined pre-post pairs:

```
syn = Synapses(pre_group, post_group, dyn1.Dynapse1SynType.AMPA, pre_list=[1,0,2], post_
↳list=[0,2,1])
```

Add synapses into network generator

We need to add synapses (which contain also the related neuron groups information) into the network generator to create a network, then convert the network into a DYNAP-SE1 configuration and apply it to the hardware.

```
# initialize a network generator
net_gen = n.NetworkGenerator()

# add synapses into the netgen
add_synapses(net_gen, syn)

# print the network so you can double check (optional)
print(net_gen.network)

# make a dynapse1config using the network
new_config = net_gen.make_dynapse1_configuration()

# apply the configuration
model.apply_configuration(new_config)
```

Create WTA

Define 2 neuron groups (note here: both of them are physical neurons instead of spike generators), specify the WTA connections between them using WTA_connections, then add the WTA into the network generator.

```
exc_group = NeuronGroup(chip1, core1, nids1)
inh_group = NeuronGroup(chip2, core2, nids2)

wta = WTA_connections(exc_group, inh_group, syn_type_ei= dyn1.Dynapse1SynType.AMPA,
syn_type_ie=dyn1.Dynapse1SynType.GABA_B, syn_type_ee=dyn1.Dynapse1SynType.NMDA, ee_pres=
[0,1,2], ee_posts=[0,1,2])

add_wta_conns(net_gen, wta)
```

Define a network using individual neurons

You can also customize your network by selecting the spike generators and neurons you want to use, and build connections between individual neurons. An example is shown below.

```
# select the spikegens you want to use
spikegen_ids = [(0,2,50)]
spikegens = []
for spikegen_id in spikegen_ids:
    spikegens.append(Neuron(spikegen_id[0],spikegen_id[1],spikegen_id[2],True))

# select the neurons you want to use
chip = 1
neuron_ids = [(chip,0,20), (0,0,36), (0,2,60), (1,1,60), (2,1,107), (2,3,152)]
neurons = []
for nid in neuron_ids:
    neurons.append(Neuron(nid[0],nid[1],nid[2]))
```

(continues on next page)

(continued from previous page)

```

# connect spikeGen to neuron0
w = 1
for i in range(w):
    net_gen.add_connection(spikegens[0], neurons[0], dyn1.Dynapse1SynType.AMPA)

# initialize a networkgenerator
net_gen = n.NetworkGenerator()

# connect neuron0 to other neurons
net_gen.add_connection(neurons[0], neurons[1], dyn1.Dynapse1SynType.AMPA)
net_gen.add_connection(neurons[0], neurons[2], dyn1.Dynapse1SynType.NMDA)
net_gen.add_connection(neurons[0], neurons[3], dyn1.Dynapse1SynType.GABA_A)
net_gen.add_connection(neurons[1], neurons[4], dyn1.Dynapse1SynType.GABA_B)
net_gen.add_connection(neurons[2], neurons[5], dyn1.Dynapse1SynType.GABA_B)

# print the network so you can double check (optional)
net_gen.print_network()

# make a dynapse1config using the network
new_config = net_gen.make_dynapse1_configuration()

# apply the configuration
model.apply_configuration(new_config)

```

1.2.4 Set up Dynapse1PoissonGen

Below is an example of how to set up one single Poisson spike generator Dynapse1PoissonGen. Here we use spike generator No.10 on the FPGA to generate Poisson spikes at 200 Hz.

```

# poissongen_id, range = [0, 1024)
poissongen_id = 10
rate = 200

# get poissongen from the model
poisson_gen = model.get_poisson_gen()
# set the target chip of the post neurons
post_chip = 0
poisson_gen.set_chip_id(post_chip)
poisson_gen.write_poisson_rate_hz(poissongen_id, rate)

# remember to start the poissongen
poisson_gen.start()

```

1.2.5 Set up Dynapse1FpgaSpikeGen

Below is an example of how to set up the FPGA spike generators Dynapse1FpgaSpikeGen using `set_fpga_spike_gen`. Here, we use spike generator No.15 on the FPGA to generate 400 spikes in 2 second with constant inter-spike interval.

```
# only use 1 spikegen No.15, [0,1024)
spikegen_id = 15
# 400 spikes in 2 second
spike_times = np.linspace(0, 2, 400)
# spikegen id list corresponding to spike_times
indices = [spikegen_id]*len(spike_times)

# the chip where the post neurons are
post_chip = 0
target_chips = [post_chip]*len(indices)
isi_base = 900
repeat_mode=False

# get the fpga spike gen from Dynapse1Model
fpga_spike_gen = model.get_fpga_spike_gen()

# set up the fpga_spike_gen
ut.set_fpga_spike_gen(fpga_spike_gen, spike_times, indices, target_chips, isi_base,
repeat_mode)

# remember to start the spikegen
fpga_spike_gen.start()
```

1.2.6 Set up parameters

Each parameter group Dynapse1ParameterGroup has 25 different parameters. A parameter Dynapse1Parameter has name, coarse value, and fine value. Below is recommended initial values of the parameters.

```
# param_name, coarse_value, fine_value

#THR, gain factor of neuron
Dynapse1Parameter("IF_THR_N", 5, 80)

#refactory period
Dynapse1Parameter("IF_RFR_N", 4, 128)

#leakage
Dynapse1Parameter("IF_TAU1_N", 4, 80)
Dynapse1Parameter("IF_TAU2_N", 0, 0)

#DC
Dynapse1Parameter("IF_DC_P", 0, 0)

#Fast excitatory synapses, AMPA
Dynapse1Parameter("NPDPIE_TAU_F_P", 4, 80)
Dynapse1Parameter("NPDPIE_THR_F_P", 0, 0)
```

(continues on next page)

(continued from previous page)

```

Dynapse1Parameter("PS_WEIGHT_EXC_F_N", 0, 0)

#Slow excitatory synapses, NMDA
Dynapse1Parameter("NPDPIE_TAU_S_P", 4, 80)
Dynapse1Parameter("NPDPIE_THR_S_P", 0, 0)
Dynapse1Parameter("PS_WEIGHT_EXC_S_N", 0, 0)
Dynapse1Parameter("IF_NMDA_N", 6, 148)

#Fast inhibitory synapses, shunting, GABA_A
Dynapse1Parameter("NPDPII_TAU_F_P", 4, 80)
Dynapse1Parameter("NPDPII_THR_F_P", 0, 0)
Dynapse1Parameter("PS_WEIGHT_INH_F_N", 0, 0)

#Slow inhibitory synapses, GABA_B
Dynapse1Parameter("NPDPII_TAU_S_P", 4, 80)
Dynapse1Parameter("NPDPII_THR_S_P", 0, 0)
Dynapse1Parameter("PS_WEIGHT_INH_S_N", 0, 0)

#adaptation
Dynapse1Parameter("IF_AHTAU_N", 4, 80)
Dynapse1Parameter("IF_AHTHR_N", 0, 0)
Dynapse1Parameter("IF_AHW_P", 0, 0)
Dynapse1Parameter("IF_CASC_N", 0, 0)

#pulse width
Dynapse1Parameter("PULSE_PWLK_P", 4, 106)

#voltage readout
Dynapse1Parameter("R2R_P", 3, 85)
Dynapse1Parameter("IF_BUF_P", 3, 80)

```

Set parameters per core

You have 3 ways to set up the parameters.

1. Generate parameter groups and apply them using :func:`update\_parameter\_group()`
<Dynapse1Model.update_parameter_group>`. Below is an example of configuration

that silence the neurons of the 16 cores in 4 chips.

```

from params import gen_clean_param_group

paramGroup = gen_clean_param_group()

for chip in range(4):
    for core in range(4):
        model.update_parameter_group(paramGroup, chip, core)

```

2. Set the parameters saved in a JSON file. Below is an example. JSON is a standard formatting style. An example JSON parameter file is [here](#).

```
# set parameters using a JSON file
ut.set_parameters_in_json_file(model, './example_parameter_files/FS_EXC_parameters.json')

# save parameters to a JSON file
# get the current config
config = model.get_configuration()
# save params
save_parameters2json_file(config, filename='./dynapse_parameters.json')
```

3. Set the parameters saved in a txt file. Below is an example. The txt format here is self-defined and more compact. An example txt parameter file is [here](#).

```
ut.set_parameters_in_txt_file(model, './example_parameter_files/parameters.txt')

# save parameters to a txt file
# get the current config
config = model.get_configuration()
# save params
save_parameters2txt_file(config, filename='./dynapse_parameters.txt')
```

Change a single parameter

If you want change only 1 single parameter value, you can use the method `update_single_parameter`.

```
param = dyn1.Dynapse1Parameter("IF_THR_N", 5, 90)
chip = 0
core = 0
model.update_single_parameter(param, chip, core)
```

1.2.7 Get spikes out of DYNAP-SE1

Usually we want monitor spikes of some specific neurons in our network. For this, we need to create a graph using `create_neuron_select_graph()` to stream the spikes out. In this graph, we connect the source node in `Dynapse1Model` to a filter node that filters the spikes of the selected neurons, then this filter node is connected to a sink node which receives the spikes and store them in its buffer.

`monitored_neurons` require neuron ids as a list of tuples (`chip_id`, `core_id`, `neuron_id`).

```
# create a graph
monitored_neurons = [(0,1,14), (0,1,15)]
graph, filter_node, sink_node = ut.create_neuron_select_graph(model, monitored_neurons)

# start the graph
graph.start()

# (optional) change the monitored neuron IDs "on the fly" if needed.
monitored_neurons = [(0,1,16)]
filter_node.set_neurons(monitored_neurons)

# clear the buffer
sink_node.get_events()
```

(continues on next page)

(continued from previous page)

```
# collect the spikes during 2 second
time.sleep(2)
# get the spikes accumulated in the buffer
events = sink_node.get_events()

# process the events (optional)
print(len(events), "events.")
for evt in events:
    ut.print_dynapse1_spike(evt)
print("")

# stop the graph
graph.stop()
```

1.2.8 Monitor membrane potential of neurons using an oscilloscope

To monitor Imem of the silicon neurons using an oscilloscope, you first get the Dynapse1Model model, then get the Dynapse1Interface api, then `monitor_neuron` can be called using this api. In each core, one neuron can be monitored at a time.

```
api = model.get_dynapse1_api()
api.monitor_neuron(chip_id, neuron_id_in_chip)
```

Note: you should use the neuron ID in the corresponding chip ranging [0, 1023), not the global neuron ID out of 4 chips.

1.2.9 Close DYNAP-SE1

You should use `close_dynapse1` at the end of your script to close the device.

If you opened DYNAP-SE1 with the GUI, this function will block the running program of your script til you click the close button of the GUI window. If you close the GUI window when your script is still running, it will NOT affect the running program, however you will not be able open the GUI again until next time you open the device.

```
ut.close_dynapse1(model, gui_process)
```

1.3 Tutorial 2: train a network on chip using STDP

Since DYNAP-SE1 doesn't have on-chip learning circuits, to train an SNN on chip we use the learning modules of this library with a computer in the loop. `stdp.py` and `stdp_utils.py` provide a learning framework for STDP-like algorithms. Implementation of triplet-STDP is given as an example, and alternative algorithms can be implemented by users. An example of training the plastic connections between a presynaptic and a postsynaptic neuron group is given in `example/test_stdp.py`.

`example/test_stdp.py` is a demo of STDP training with 10 samples. The samples have the same value. The learned weight matrix should be similar to .. code-block:

```
w = [[ 1  1  1]
      [ 0 10  0]
      [ 1  1  1]]
```

i.e. `w[1][1]` has the strongest weights because pre neuron1 and post neuron1 receive the strongest stimulation from the spike generators and thus fire the most.

A network consisting of a spike generator group of 3 spike generators, a pre and a post neuron group with 3 neurons, respectively is built on chip using the code below. The spike generators are connected to both pre and post neuron groups with one-to-one connections. The pre neuron group is connected to the post one with the initial weight matrix `int_w_plast`.

```
spikegen_group = NeuronGroup(schip,score,sids,True)

pre_neuron_group = NeuronGroup(chip,core,pre_nids)
post_neuron_group = NeuronGroup(chip,core,post_nids)

# connect spikegen_group to pre and post neuron_group, connect pre to post
connectivity = {
    'pre_gen2pre': Synapses(spikegen_group, pre_neuron_group, dyn1.Dynapse1SynType.
        NMDA, conn_type='one2one', mux_conn=mux_conn_spikegen),
    'post_gen2post': Synapses(spikegen_group, post_neuron_group, dyn1.Dynapse1SynType.
        NMDA, conn_type='one2one', mux_conn=mux_conn_spikegen),
    'pre2post': Synapses(pre_neuron_group, post_neuron_group, dyn1.Dynapse1SynType.
        AMPA, weight_matrix=int_w_plast)
}

for conn in connectivity:
    add_synapses(net_gen, connectivity[conn])

new_config = net_gen.make_dynapse1_configuration()
model.apply_configuration(new_config)
```

The STDP module is instantiated using `Sdp` class.

```
stdp = Sdp(model, net_gen, pre_neuron_ids, post_neuron_ids, w_plast, stdp_param_file,
    algorithm=algorithm, new_thread=stdp_new_thread)
```

`stdp.start_stdp()` starts the learning. Then 10 training samples (with the same input) are fed to the SNN in a loop:

```
for sample in range(num_samples):
    # give new stimulation
    for i in range(len(global_poisson_gen_ids)):
        poisson_gen.write_poisson_rate_hz(global_poisson_gen_ids[i], rates[i])
    poisson_gen.start()

    # learn: w_plast being updated in another thread
    time.sleep(float(duration_per_sample/1e3))

    poisson_gen.stop()

    # remove the current pre post connections
    remove_synapses(net_gen, connectivity['pre2post'])

    # add new pre-post connections using the latest w_plast
    current_w_plast = stdp.w_plast
    int_w_plast = floatW2intW(current_w_plast, max_pre_count)
    connectivity['pre2post'] = Synapses(pre_neuron_group, post_neuron_group, dyn1.
    Dynapse1SynType.AMPA, weight_matrix=int_w_plast)
```

(continues on next page)

(continued from previous page)

```

add_synapses(net_gen, connectivity['pre2post'])

print(current_w_plast)
print(int_w_plast)

new_config = net_gen.make_dynapse1_configuration()
model.apply_configuration(new_config)

# cooling down
time.sleep(float(duration_cool_down/1e3))

stdp.stop_stdp()

```

After each training sample lasting for `duration_per_sample`, the updated `stdp.w_plast` maintained by the STDP module is applied to the hardware. To change the connections inside the network generator, the old 'pre2post' connections are first removed using `remove_synapses(net_gen, connectivity['pre2post'])`, then the new `int_w_plast` discretized using `stdp.w_plast` are added to the network generator using `add_synapses(net_gen, connectivity['pre2post'])`. Note that the `remove_synapses` and `add_synapses` are only operated inside the network generator, not the hardware. The new DYNAP-SE1 configuration has to be generated using ```net_gen.make_dynapse1_configuration()``, and applied to the hardware.

Since the rates of the spike generators are set to `rates = [0, 200, 0]`, i.e. the middle neuron in each neural population receives the strongest stimulation, the synapse between this pair of neurons grows to be the strongest one.

1.4 Modules

1.4.1 dynapse1utils

dynapse1utils.py contains functions that wraps around low-level Samna APIs for DYNAP-SE1 users.

`dynapse1utils.close_dynapse1(model, gui_process="")`

Close DYNAP-SE1 board with or without the GUI.

Parameters

- **model** (*Dynapse1Model*) – the DYNAP-SE1 model you get when you [open_dynapse1](#).
- **gui_process** (*Process*) – the GUI process handler you created when you [open_dynapse1](#).

`dynapse1utils.create_neuron_select_graph(model, neuron_ids)`

Create a graph: `source_node` in `model` -> `filter_node` in `graph` -> `sink_node` to get events. Only `filter_node` is in the graph. Source and sink nodes are outside graph.

Usage: To use the created/returned graph, first `graph.start()`. To get events, `sink_node.get_events()`. If you `graph.stop()`, for now the graph actually won't stop, all events are still streamed into the buffer of `sink_node`. This is work in progress. Thus to get events for 1 second, you need to first clear the buffer of `sink_node` using `get_events()`. i.e., .. code-block:

```

sink_node.get_events()
sleep(1)
events = sink_node.get_events()

```

See details at: <https://symsense-sys-int.gitlab.io/samna/filters.html#>

Parameters

- **model** (*Dynapse1Model*) – , returned by `open_dynapse1()`
- **neuron_ids** (*list[tuple(int, int, int)]*) – tuple in the order of (chip, core, neuron), neuron ids of the neurons you want to monitor.

Returns

- **graph** (*samna.graph.EventFilterGraph*) – samna filtering graph.
- **filter_node** (*samna.graph.nodes.Dynapse1NeuronSelect_dynapse1_event*) – filter_node that filters the events of selected neurons.
- **sink_node** (*samna.BasicSinkNode_dynapse1_dynapse1_event*) – sink node that receives the events from the filter node.

dynapse1utils.free_port()

Get a free port number using sockets.

dynapse1utils.gen_destination_string(destination)

Convert a *Dynapse1Destination* in the string format for printing: C<target_chip_id>c<core_mask><in_use>. e.g., C2c1True means this *Dynapse1Destination* has target_chip_id=2, core_mask=1 and it's occupied.

Parameters **destination** (*Dynapse1Destination*) – destination.

dynapse1utils.gen_synapse_string(synapse)

Convert a *Dynapse1Synapse* in the string format for printing: c<listen_core_id>n<listen_neuron_id><synapse_type>. e.g., c1n25NMDA means this synapse listens to core 1 neuron 25 with type NMDA.

Parameters **synapse** (*Dynapse1Synapse*) – synapse.

dynapse1utils.get_Dynapse1_viz_converter()

Helper function to reuse viz node for DYNAP-SE1 GUI.

dynapse1utils.get_global_id(chip, core, neuron)

Conveniently generate the corresponding global neuron ID given chip, core, neuron IDs.

Parameters

- **chip** (*int*) – chip ID, [0,4).
- **core** (*int*) – core ID, [0,4) for physical neurons, [0,1) for spike generators on the FPGA.
- **neuron** (*int*) – neuron ID, [0,256).

Returns `global_neuron_id`.

Return type `int`.

dynapse1utils.get_global_id_list(tuple_list)

Conveniently generate a list of global neuron IDs given a list of [chip, core, neuron ID] tuples.

Parameters **tuple_list** (*list[[int, int, int]]*) – a list of global neuron IDs.

Returns `global_neuron_ids`.

Return type `list[int]`.

dynapse1utils.get_neuron_from_config(config, chip, core, neuron)

Get a neuron by its `global_neuron_id` from a configuration.

Parameters

- **config** (*Dynapse1Configuration*) – *Dynapse1Configuration*.

- **global_neuron_id** (*int*) – global neuron id, [0,4096).

Returns Dynapse1Neuron.

Return type samna.dynapse1.Dynapse1Neuron().

`dynapse1utils.get_parameter_value(config, chip, core, name)`

Get the parameter (coarse, fine) value of a specific parameter in the DYNAP-SE1 configuration.

Parameters

- **config** (*samna.dynapse1.Dynapse1Configuration*) – Dynapse1Configuration
- **chip** (*int*) – chip ID.
- **core** (*int*) – core ID.
- **name** (*str*) – parameter name.

Returns tuple (*int,int*) – (coarse, fine) value.

`dynapse1utils.get_parameters(config, chip, core)`

Get a list of Dynapse1Parameter (25 in total) of a specific core.

Parameters

- **config** (*Dynapse1Configuration*) – Dynapse1Configuration retrieved from Dynapse1Model.
- **chip** (*int*) – chip ID, [0,4).
- **core** (*int*) – core ID, [0,4).

Returns current parameter setup of the core.

Return type list[samna.dynapse1.Dynapse1Parameter].

`dynapse1utils.get_selected_timestamps(spikes, neuron_ids)`

Given Dynapse1Spikes retrieved from NeuronSelect sinknode, filter out timestamps of selected neuron_ids.

Parameters

- **spikes** (*list[Dynapse1Spike]*) – result of sink_node.get_events()
- **neuron_ids** (*list[(int,int,int)]*) – neuron_ids

Returns timestamps (*list[list[int]]*) – spike timestamps of selected neurons. The list indexing is the same as the neuron indexing in the list neuron_ids.

`dynapse1utils.get_selected_traces(timed_traces, neuron_ids)`

Filter out timestamps and trace_values of selected neuron_ids.

Parameters

- **timed_traces** (*list[Dynapse1Trace]*) – result of sink_node.get_events()
- **neuron_ids** (*list[(int,int,int)]*) – neuron_ids

Returns

- **timestamps** (*list[list[int]]*) – spike timestamps of selected neurons. The list indexing is the same as the neuron indexing in the list neuron_ids.
- **trace_values** (*list[list[float]]*) – trace values at spike times. The list indexing is the same as the neuron indexing in the list neuron_ids.

`dynapse1utils.get_serial_number(device_idx=0, select_device=False)`

Get serial number of an opened DYNAP-SE1 board.

Parameters

- **device_idx** (*int, optional*) – DYNAP-SE1 board index. Defaults to 0.
- **select_device** (*bool, optional*) – whether to select one DYNAP-SE1 board out of multiple connected ones. Defaults to False.

Returns *int* – serial_number

`dynapse1utils.get_time_wrap_events(model)`

DYNAP-SE1 sends out 2 types of events: spike and timeWrapEvent. timeWrapEvent occurs when the 32 bit timestamp wraps around. This happens every ~37(?) minutes. With the graph created by this method, you can monitor if any timeWrapEvent is generated effectively with the C++ backend.

Agrs: *model* (`samna.dynapse1.Dynapse1Model`): returned by `open_dynapse1()`

`dynapse1utils.get_trace_value(traces, timestamp)`

Get 1 single trace value from a list of `Dynapse1Traces`.

Parameters

- **traces** (*list[samna.dynapse1.Dynapse1Trace]*) – list of trace events.
- **timestamp** (*int*) – DYNAP-SE1 timestamp, in microsecond.

Returns `samna.dynapse1.Dynapse1Trace` – the trace with timestamp. If not found, return `None`.

`dynapse1utils.open_device(sender_port=33336, receiver_port=33335, select_device=False)`

Get unopened devices detected by samna.

Parameters

- **sender_port** (*int*) – samnaNode's sending port.
- **receiver_port** (*int*) – samnaNode's receiving port.
- **select_device** (*bool*) – Defaults to False.

Returns

- **device** – Samna device model
- **samna_info_dict** (*dictionary*) – information dictionary.

`dynapse1utils.open_dynapse1(gui=True, select_device=False, sender_port=33336, receiver_port=33335)`

open DYNAP-SE1 board with or without GUI. Note that input parameter `device_name` has been deprecated now, please do not assign `device_name` anymore! You cannot name DYNAP-SE1 board by yourself because it's now assigned as 'Dynapse1DevKit:index' by `open_dynapse1()`

Parameters

- **gui** (*bool*) – whether to open the gui or not. "True" will return store and `gui_process`; "False" will only return store.
- **select_device** (*bool*) – whether to select one DYNAP-SE1 board out of some connected ones. "False": board 0 will be opened; "True": user will be asked to choose the board index
- **sender_port** (*int*) – samnaNode's sending port. Should be 33336 if `gui=True`, otherwise you can use a 5-digits port number.
- **receiver_port** (*int*) – samnaNode's receiving port. Should be 33335 if `gui=True`, otherwise you can use a 5-digits port number which is different from `sender_port`.

Returns Dynapse1Model: model, DYNAP-SE1 model of the selected device. gui_process: the gui process handler if gui=True. Otherwise, an empty string ''.

`dynapse1utils.open_gui(device, visualizer_id=3)`

Open DYNAP-SE1 spiking GUI.

`dynapse1utils.open_visualizer(window_width, window_height, receiver_endpoint, sender_endpoint, visualizer_id)`

start visualizer in a isolated process which is required on mac, instead of a sub process. It will not return until the remote node is opened. Return the opened visualizer.

`dynapse1utils.print_dynapse1_spike(event)`

Print the Spike in the following format: "(timestamp, global_neuron_id)".

Parameters `event` (`samna.dynapse1.Spike`) – spike.

`dynapse1utils.print_neuron_destinations(neuron, destination_id_list=range(0, 4))`

Print the destinations of the neuron given the destination id list.

Parameters

- **neuron** (`Dynapse1Neuron`) – neuron.
- **destination_id_list** (`list[int]`) – destination IDs you want to print out.

`dynapse1utils.print_neuron_synapses(neuron, synapse_id_list=range(0, 64))`

Print the synapses of the neuron given the synapse id list.

Parameters

- **neuron** (`Dynapse1Neuron`) – neuron.
- **synapse_id_list** (`list[int]`) – synapse IDs you want to print out.

`dynapse1utils.save_parameters2json_file(config, filename='./dynapse_parameters.json')`

Save the parameters of 16 cores into a JSON file.

Parameters

- **config** (`Dynapse1Configuration`) – Dynapse1Configuration retrieved from Dynapse1Model.
- **filename** (`string`) – the path and filename of the output parameter file.

`dynapse1utils.save_parameters2txt_file(config, filename='./dynapse_parameters.txt')`

Save the parameters of 16 cores into a txt file.

Parameters

- **config** (`Dynapse1Configuration`) – Dynapse1Configuration retrieved from Dynapse1Model.
- **filename** (`string`) – the path and filename of the output parameter file.

`dynapse1utils.set_fpga_spike_gen(fpga_spike_gen, spike_times, indices, target_chips, isi_base, repeat_mode=False)`

Author: Nicoletta Risi. Adapted by Jingyue Zhao.

This function sets the SpikeGenerator object given a list of spike times, in sec, correspondent input neuron ids and the target chips, i.e. the chip destination of each input event.

About isi_base: Given a list of spike times (in sec) a list of isi (Inter Stimulus Interval) is generated. Given a list of isi, the resulting list of isi set from the FPGA will be:

$isi * unit_fpga$ with $unit_fpga = isi_base/90 * us$

Thus, given a list of spike_times in sec:

- first the spike times are converted in us
- then a list of isi (in us) is generated
- **then the list of isi is divided by the unit_fpga (so that the** resulting list of isi set on FPGA will have the correct unit given the input isi_base)

E.g.: if isi_base=900 the list of generated isi will be multiplied on FPGA by $900/90 us = 10 us$

Parameters

- **fpga_spike_gen** (*samna.dynapse1.Dynapse1FpgaSpikeGen*) – Dynapse1FpgaSpikeGen retrieved from Dynapse1Model.
- **spike_times** (*list*) – list of input spike times, in second.
- **indices** (*list*) – list of FPGA spike generator ids sorted according to time of spike.
- **target_chips** (*list*) – list of target chip to which each event will be sent.
- **isi_base** (*int*) – 90 or 900 (See above)
- **repeat_mode** (*bool*) – If repeat is True, the spike generator will loop from the beginning when it reaches the end of the stimulus.

`dynapse1utils.set_parameters_in_json_file(model, filename='./dynapse_parameters.json')`

Load the parameters in a JSON file to DYNAP-SE1 board.

Parameters

- **model** (*Dynapse1Model*) – Dynapse1Model.
- **filename** (*string*) – the path and filename of the input parameter file.

`dynapse1utils.set_parameters_in_txt_file(model, filename='./dynapse_parameters.txt')`

Load the parameters in a txt file to DYNAP-SE1 board.

Parameters

- **model** (*Dynapse1Model*) – Dynapse1Model.
- **filename** (*string*) – the path and filename of the input parameter file.

1.4.2 netgen

netgen.py includes basic network elements, e.g. neuron groups, synapses and Winner-Take-All (WTA) structures, that can be added into NetworkGenerator to implement Spiking Neural Networks (SNNs) on DYNAP-SE1 processor.

class netgen.Network

Bases: object

Network that checks and maintains the topology of a spiking neural network that meets DYNAP-SE1 hardware constraints.

post_neuron_dict

a dictionary which stores all the post neurons (and their incoming connections).

- **key: tuple, (post.chip_id, post.core_id).** Divide the post neurons by its location (core) for aliasing check.
- **value:** list of neurons each of which has incoming connections.

Type dictionary

add_connection(*pre, post, synapse_type*)

Add a connection between the (pre,post) neuron pair and checks if the new connection meet DYNAP-SE1 hardware constraints. Raise warnings or exceptions if something goes wrong.

Parameters

- **pre** (*netgen.Neuron*) – presynaptic neuron.
- **post** (*netgen.Neuron*) – postsynaptic neuron.
- **synapse_type** (*samna.dynapse1.SynType*) – synapse type.

Raises **Exception** – post neuron cannot be a spike generator.

add_connections_from_list(*pre_neurons, post_neurons, synapse_type, pre_ids, post_ids*)

Add multiple connections between 2 neuron groups.

Parameters

- **pre_neurons** (*list[netgen.Neuron]*) – list of neuron objects.
- **post_neurons** (*list[netgen.Neuron]*) – list of neuron objects.
- **synapse_type** (*samna.dynapse1.SynType*) – synapse type.
- **pre_ids** (*list[int]*) – neuron indices in pre_neurons list.
- **post_ids** (*list[int]*) – neuron indices in post_neurons list.

add_connections_from_type(*pre_neurons, post_neurons, synapse_type, conn_type, p=1.0, rand_seed=None*)

Add multiple connections between 2 neuron groups given connectivity type: ‘one2one’ or ‘all2all’.

Parameters

- **pre_neurons** (*list[netgen.Neuron]*) – list of neuron objects.
- **post_neurons** (*list[netgen.Neuron]*) – list of neuron objects.
- **synapse_type** (*samna.dynapse1.SynType*) – synapse type.
- **conn_type** (*str*) – ‘one2one’ or ‘all2all’.
- **p** (*float, optional*) – possibility of all2all connections. Defaults to 1.0.
- **rand_seed** (*int, optional*) – random seed for all2all connections. Defaults to None.

remove_connection(*pre, post, synapse_type*)

Remove connection between the (pre,post) neuron pair. If the (pre, post) neuron pair only has one connection, the neurons will be removed from the network as well. Otherwise only this connection is removed.

Parameters

- **pre** (*netgen.Neuron*) – presynaptic neuron.
- **post** (*netgen.Neuron*) – postsynaptic neuron.
- **synapse_type** (*samna.dynapse1.SynType*) – synapse type.

Raises

- **Exception** – post neuron cannot be a spike generator.
- **Exception** – connection that does not exist in the network cannot be remove.

- **Exception** – connection that does not exist in the network cannot be remove.

remove_connections_from_list(*pre_neurons*, *post_neurons*, *synapse_type*, *pre_ids*, *post_ids*)

Remove multiple connections between 2 neuron groups

Parameters

- **pre_neurons** (*list*[[netgen.Neuron](#)]) – list of neuron objects.
- **post_neurons** (*list*[[netgen.Neuron](#)]) – list of neuron objects.
- **synapse_type** (*samna.dynapse1.SynType*) – synapse type.
- **pre_ids** (*list*[*int*]) – neuron indices in pre_neurons list.
- **post_ids** (*list*[*int*]) – neuron indices in post_neurons list.

class netgen.NetworkGenerator

Bases: object

NetworkGenerator maintains a Network and a corresponding Dynapse1Configuration. A NetworkGenerator must be used to convert you network topology added to the NetworkGenerator into a Dynapse1Configuration that can be applied to hardware.

Whenever you want to change your network, you need to add/remove connections using the previous NetworkGenerator from which you took the configuration and applied it using the model during your last modification of the network. Otherwise, you just create a new NetworkGenerator and configuration from the scratch and apply it.

config

hardware applicable DYNAP-SE1 configuration converted from the network stored in the NetworkGenerator.

Type *samna.dynapse1.Dynapse1Configuration*

network

network description of neurons and connections.

Type *netgen.Network*

add_connection(*pre*, *post*, *synapse_type*)

Add a connection between the pre and post neuron with type “synapse_type” into the network.

Parameters

- **pre** ([netgen.Neuron](#)) – presynaptic neuron.
- **post** ([netgen.Neuron](#)) – postsynaptic neuron.
- **synapse_type** (*samna.dynapse1.SynType*) – synapse type.

add_connections_from_list(*pre_neurons*, *post_neurons*, *synapse_type*, *pre_ids*, *post_ids*)

Add multiple connections between 2 neuron groups.

Parameters

- **pre_neurons** (*list*[[netgen.Neuron](#)]) – list of neuron objects.
- **post_neurons** (*list*[[netgen.Neuron](#)]) – list of neuron objects.
- **synapse_type** (*samna.dynapse1.SynType*) – synapse type.
- **pre_ids** (*list*[*int*]) – neuron indices in pre_neurons list.
- **post_ids** (*list*[*int*]) – neuron indices in post_neurons list.

add_connections_from_type(*pre_neurons, post_neurons, synapse_type, conn_type, p=1, rand_seed=None*)

Add multiple connections between 2 neuron groups given connectivity type: 'one2one' or 'all2all'.

Parameters

- **pre_neurons** (*list[netgen.Neuron]*) – list of neuron objects.
- **post_neurons** (*list[netgen.Neuron]*) – list of neuron objects.
- **synapse_type** (*samna.dynapse1.SynType*) – synapse type.
- **conn_type** (*str*) – 'one2one' or 'all2all'.
- **p** (*float, optional*) – possibility of all2all connections. Defaults to 1.0.
- **rand_seed** (*int, optional*) – random seed for all2all connections. Defaults to None.

clear_network()

Clear all neurons and connections added to the network.

make_dynapse1_configuration(*validation=True*)

Convert network defined in the network generator into a DYNAP-SE1 applicable configuration. Checks if the network of the network generator is valid or not, i.e. meets DYNAP-SE1 hardware constraints. If valid, network will be converted to a DYNAP-SE1 configuration. Otherwise, exceptions will be raised.

Parameters **validation** (*bool, optional*) – whether to validate the network before converting to a DYNAP-SE1 configuration. Defaults to True.

Returns *samna.dynapse1.Dynapse1Configuration* – Dynapse1Configuration.

make_dynapse1_configuration_in_chip(*chip_id*)

Convert network defined in the network generator into a DYNAP-SE1 applicable configuration which only lies in one target chip. The configuration of the remaining 3 chips will not be changed.

This function should be followed by `apply_configuration_by_chip()` which applies the new configuration only to a single chip of the DYNAP-SE1 board.

Note: Be careful if you use this method because this is specifically designed for the user who only does experiments in a single chip. If you have network components in other chips, you will get an error.

Parameters **chip_id** (*int*) – chip id, [0,4).

Raises **Exception** – network has neuron(s) outside the target chip.

Returns *samna.dynapse1.Dynapse1Configuration* – Dynapse1Configuration.

make_dynapse1_configuration_in_core(*chip_id, core_id*)

Convert network defined in the network generator into a DYNAP-SE1 applicable configuration which only lies in one target core. The configuration of the remaining 15 cores will not be changed.

This function should be followed by `apply_configuration_by_core()` which applies the new configuration only to a single chip of the DYNAP-SE1 board.

Note: Be careful if you use this method because this is specifically designed for the user who only does experiments in a single core. If you have network components in other core, you will get an error.

Parameters

- **chip_id** (*int*) – chip id, [0,4).
- **core_id** (*int*) – core id, [0,4).

Raises Exception – network has neuron(s) outside the target core.

Returns `samna.dynapse1.Dynapse1Configuration` – Dynapse1Configuration.

print_network()

Warning: `print_network` is deprecated and will be removed in a future release, use `print(NetworkGenerator.network)`. Note: `str(NetworkGenerator.network)` gives you the string format of a network.

remove_connection(*pre, post, synapse_type*)

Remove a connection between the pre and post neuron with type “synapse_type” into the network.

Parameters

- **pre** (`netgen.Neuron`) – presynaptic neuron.
- **post** (`netgen.Neuron`) – postsynaptic neuron.
- **synapse_type** (`samna.dynapse1.SynType`) – synapse type.

remove_connections_from_list(*pre_neurons, post_neurons, synapse_type, pre_ids, post_ids*)

Remove multiple connections between 2 neuron groups

Parameters

- **pre_neurons** (`list[netgen.Neuron]`) – list of neuron objects.
- **post_neurons** (`list[netgen.Neuron]`) – list of neuron objects.
- **synapse_type** (`samna.dynapse1.SynType`) – synapse type.
- **pre_ids** (`list[int]`) – neuron indices in pre_neurons list.
- **post_ids** (`list[int]`) – neuron indices in post_neurons list.

class netgen.Neuron(*chip_id=0, core_id=0, neuron_id=0, is_spike_gen=False*)

Bases: object

Neuron object used to create network topology inside a network generator, Not the silicon neuron on DYNAP-SE1 hardware.

Parameters

- **chip_id** (*int*) – [0,4).Defaults to 0.
- **core_id** (*int*) – [0,4).Defaults to 0.
- **neuron_id** (*int*) – [0,256).Defaults to 0.
- **is_spike_gen** (*bool*) – True if this neuron is a spike generator on the FPGA, otherwise a physical neuron on chip. Defaults to False.

chip_id

Defaults to 0.

Type int

core_id

Defaults to 0.

Type int

neuron_id

Defaults to 0.

Type int

is_spike_gen

True if this neuron is a spike generator on the FPGA, otherwise a physical neuron on chip. Defaults to False.

Type bool

incoming_connections

a dictionary which stores the incoming_connections. Only starts to play a role after add_connection(pre, neuron).

- **key (tuple): (pre.core_id, pre.neuron_id, synapse_type). Corresponds to cam.** Divide the connections by its cam value for cam reuse.
- **value (list): [(pre.chip_id, pre.is_spike_gen), (pre.chip_id, pre.is_spike_gen), ...].** To tell if the post neurons are the same neuron. get the connection weight.

Type dictionary

class netgen.NeuronGroup(chip_id=0, core_id=0, neuron_ids=None, is_spike_gen=False)

Bases: object

A collection of netgen.Neurons for neural population operations.

Parameters

- **chip_id** (int) – chip id, defaults to 0.
- **core_id** (int) – core id, defaults to 0.
- **neuron_ids** (list[int]) – list of neuron ids, defaults to None.
- **is_spike_gen** (bool, defaults to False.) – True if this neuron group is a group of spike generators on the FPGA, otherwise silicon neurons on chip.

chip_id

chip id

Type int

core_id

core id

Type int

neuron_ids

list of neuron ids

Type list[int]

is_spike_gen

if this neuron group is a group of spike generators on the FPGA or silicon neurons on chip.

Type bool

property neurons

neurons getter function to get neurons according to the current chip/core/neuron ids of the neuron group.

Returns list[netgen.Neuron] – A list of neurons given the current chip/core/neuron ids.

property tuple_neuron_ids

list[(chip, core, neuron)] getter function

Returns list[netgen.Neuron] – A list of tuple neuron ids given the current chip/core/neuron ids

class netgen.Synapses(*pre_group, post_group, synapse_type, pre_list=None, post_list=None, conn_type=None, p=None, rand_seed=None, weight_matrix=None, mux_conn=1*)

Bases: object

Stores the information of the connectivity from a pre NeuronGroup to a post NeuronGroup. The connections can be defined in 3 ways:

- using self-defined pre_list and post_list
- using connectivity pattern, conn_type
- using weight_matrix

Parameters

- **pre** (NeuronGroup) – presynaptic neuron group.
- **post** (NeuronGroup) – postsynaptic neuron group.
- **synapse_type** (samna.dynapse1.Dynapse1SynType) – synapse type.
- **pre_list** (list[int]) – the relative index of neurons in the pre_group, not the (chip, core, neuron) id. Defaults to None.
- **post_list** (list[int]) – the relative index of neurons in the post_group, not the (chip, core, neuron) id. Defaults to None.
- **conn_type** (str) – in ['one2one', 'all2all']. Defaults to None.
- **p** (float) – possibility of all2all random connections. Defaults to None.
- **rand_seed** (int) – random seed of all2all random connections. Defaults to None.
- **mux_conn** (int) – the multiplexer number of the connections. Each pre-post synapse will be mux_conn connections. Defaults to 1.

pre_group

presynaptic neuron group.

Type NeuronGroup

post_group

postsynaptic neuron group.

Type NeuronGroup

synapse_type

synapse type.

Type samna.dynapse1.Dynapse1SynType

pre_list

the relative index of neurons in the pre_group, not the (chip, core, neuron) id. Defaults to None.

Type list[int]

post_list

the relative index of neurons in the post_group, not the (chip, core, neuron) id. Defaults to None.

Type list[int]

conn_type

in ['one2one', 'all2all']. Defaults to None.

Type str

p

possibility of all2all random connections. Defaults to None.

Type float

rand_seed

random seed of all2all random connections. Defaults to None.

Type int

weight_matrix

weight matrix between pre_group and post_group. Rows represent pre_ids, columns represent post_ids. Defaults to None.

Type numpy.array()

mux_conn

the multiplexer number of the connections. Each pre-post synapse will be mux_conn connections. Defaults to 1.

Type int

```
class netgen.WTA_connections(exc_group, inh_group, syn_type_ei, syn_type_ie, syn_type_ee=None, p_ei=1,
                             p_ie=1, ee_pres=None, ee_posts=None, rand_seed=None, mux_conn_ei=1,
                             mux_conn_ie=1, mux_conn_ee=1)
```

Bases: object

Define EI IE EE connections of a WTA with an excitatory and an inhibitory population.

ei

EI connections.

Type *netgen.Synapses*

ie

IE connections.

Type *netgen.Synapses*

ee

EE connections. Defaults to None.

Type *netgen.Synapses*

Parameters

- **exc_group** (*NeuronGroup*) – excitatory neuron group.
- **inh_group** (*NeuronGroup*) – inhibitory neuron group.
- **syn_type_ei** (*samna.dynapse1.SynType*) – EI synapse type.
- **syn_type_ie** (*samna.dynapse1.SynType*) – IE synapse type.

- **syn_type_ee** (*samna.dynapse1.SynType*) – recurrent excitatory (EE) synapse type inside excitatory neurons. Defaults to None.
- **p_ei** (*float*) – probability of EI connections.
- **p_ie** (*float*) – probability of IE connections.
- **ee_pres** (*list[int]*) – pre_index_list of the EE connections. Defaults to None.
- **ee_posts** (*list[int]*) – post_index_list of the EE connections. Defaults to None.
- **rand_seed** (*int*) – random seed for EI and IE connections. Defaults to None.
- **mux_conn_ei** (*int*) – multiplexer of EI connections.
- **mux_conn_ie** (*int*) – multiplexer of IE connections.
- **mux_conn_ee** (*int*) – multiplexer of EE connections.

netgen.add_synapses(*netgen, synapse*)

Add a Synapses object into a network generator.

Parameters

- **netgen** (*netgen.NetworkGenerator*) – NetworkGenerator.
- **synapse** (*netgen.Synapses*) – synapse that contains pre, post neurons and connections between them.

netgen.add_wta_conns(*netgen, wta_conns*)

Add WTA connections to create a WTA in the network generator.

Parameters

- **netgen** (*netgen.NetworkGenerator*) – NetworkGenerator.
- **wta_conns** (*netgen.WTA_connections*) – WTA_connections that contains excitatory, inhibitory neurons and WTA connections between them.

netgen.print_post_neuron_dict(*post_neuron_dict*)

Print post_neuron_dict of the Network.

Parameters **post_neuron_dict** (*dictionary*) – post_neuron_dict.

netgen.remove_synapses(*netgen, synapse*)

Remove a synapse group from a network generator. If a neuron pair (pre, post) doesn't have any connections after this remove operation, both of the neurons will be removed from the network generator.

Parameters

- **netgen** (*netgen.NetworkGenerator*) – NetworkGenerator.
- **wta_conns** (*netgen.synapse*) – synapse that contains pre, post neurons and connections between them.

1.4.3 params

params.py gives some board parameter configurations as examples.

`params.gen_clean_param_group()`

Generate a Dynapse1ParameterGroup of one core which should be able to silence the neurons.

Returns `samna.dynapse1.Dynapse1ParameterGroup` – Dynapse1ParameterGroup.

`params.gen_dc_params()`

Generate a Dynapse1ParameterGroup based on silent neurons, and turned DC current on.

Returns `samna.dynapse1.Dynapse1ParameterGroup` – Dynapse1ParameterGroup.

`params.gen_param_group()`

Generate a Dynapse1ParameterGroup of one core with some synapse weights turned on for examples.

Returns `samna.dynapse1.Dynapse1ParameterGroup` – Dynapse1ParameterGroup.

`params.gen_stdp_params()`

Generate a Dynapse1ParameterGroup for STDP example.

Returns `samna.dynapse1.Dynapse1ParameterGroup` – Dynapse1ParameterGroup.

`params.set_params(model, dc=False, param_group=None)`

Set 16 DYNAP-SE1 cores with the same Dynapse1ParameterGroup for examples.

Parameters

- **model** (`samna.dynapse1.Dynapse1Model`) – Dynapse1Model
- **dc** (*bool*, *optional*) – Turn DC on if True. Defaults to False.
- **param_group** (`samna.dynapse1.Dynapse1ParameterGroup`, *optional*) – Dynapse1ParameterGroup can be specified. Defaults to None.

`params.set_stdp_params(model)`

Set 16 DYNAP-SE1 cores with the same Dynapse1ParameterGroup for STDP example.

Parameters **model** (`samna.dynapse1.Dynapse1Model`) – Dynapse1Model

1.4.4 STDP learning

stdp.py and *stdp_utils.py* provide a learning framework for STDP-like algorithms. Implementation of triplet-STDP is given as an example, and other algorithms can be implemented by users. An example of training the plastic connections between a presynaptic and a postsynaptic neuron group is in *example/test_stdp.py*.

stdp

```
class stdp.Stdp(model, net_gen, pre_neuron_ids, post_neuron_ids, w_plast, param_file,
                algorithm='triplet_stdp', new_thread=True, remove_bad_traces=False, stop_graph=False,
                spike_sink_debug=False, max_trace_num=10, max_time_interval=300000.0)
```

Bases: object

A class which implements STDP-like learning framework between a pre neuron population and a post neuron population with a computer in the loop.

Parameters

- **model** (*samna.dynapse1.Dynapse1Model*) – Dynapse1Model you use to configure DYNAP-SE1 board.
- **net_gen** (*netgen.NetworkGenerator*) – NetworkGenerator you use to change the network connections.
- **pre_neuron_ids** (*list[tuple(int,int,int)]*) – presynaptic neuron IDs (chip, core, neuron).
- **post_neuron_ids** (*list[tuple(int,int,int)]*) – postsynaptic neuron IDs (chip, core, neuron).
- **w_plast** (*numpy.array*) – initial weight matrix.
- **param_file** (*str*) – json filename that store the STDP algorithm parameters.
- **algorithm** (*str, optional*) – learning algorithm name. Defaults to ‘triplet_stdp’. ‘triplet_stdp’ is implemented, and other STDP-like learning algorithms can be implemented by users.
- **new_thread** (*bool, optional*) – True if you want to start a new thread for STDP backend running. Because you may need another thread to change network connections. Defaults to True.
- **remove_bad_traces** (*bool, optional*) – whether to drop abnormal traces during learning. Defaults to False.
- **stop_graph** (*bool, optional*) – whether to stop filtering graph when stop_stdp. Defaults to False.
- **spike_sink_debug** (*bool, optional*) – whether to add spike filters to debug trace filters. Defaults to False.
- **max_num** (*int, optional*) – number of received traces at one get_events(). Defaults to 10.
- **max_time_interval** (*int, optional*) – time difference between the traces, in microsecond. Defaults to 3*1e5.

model

Dynapse1Model you use to configure DYNAP-SE1 board.

Type *samna.dynapse1.Dynapse1Model*

net_gen

NetworkGenerator you use to change the network connections.

Type *netgen.NetworkGenerator*

pre_neuron_ids

presynaptic neuron IDs (chip, core, neuron).

Type *list[tuple(int,int,int)]*

post_neuron_ids

postsynaptic neuron IDs (chip, core, neuron).

Type *list[tuple(int,int,int)]*

w_plast

plastic weight matrix updated over time during learning.

Type *numpy.array*

param_file

json filename that store the STDP algorithm parameters.

Type str

algorithm

learning algorithm name. Defaults to 'triplet_stdp'. 'triplet_stdp' is implemented, and other STDP-like learning algorithms can be implemented by users.

Type str

new_thread

True if you want to start a new thread for STDP backend running. Because you may need another thread to change network connections. Defaults to True.

Type bool

remove_bad_traces

whether to drop abnormal traces during learning. Defaults to False.

Type bool

stop_graph

whether to stop filtering graph when stop_stdp. Defaults to False.

Type bool

spike_sink_debug

whether to add spike filters to debug trace filters. Defaults to False.

Type bool

max_trace_num

max_trace_num of bad traces. Defaults to 10.

Type int, optional

max_time_interval

max_time_interval of bad traces. Defaults to 3*1e5, in microsecond.

Type int, optional

graph

samna filtering graph.

Type samna.graph.EventFilterGraph

nodes (dictionary{'str'

samna_nodes})): if spike_sink_debug is False, nodes contains .. code-block:

```
nodes = {
    'spike_filter': spike_filter_node,
    'onpre_trace_filter': onpre_trace_node,
    'onpost_trace_filter': onpost_trace_node,
    'onpre_trace_sink': onpre_trace_sink,
    'onpost_trace_sink': onpost_trace_sink
}
```

Otherwise, nodes also has the following elements: .. code-block:

```
spike_nodes = {
    'pre_spike_filter': pre_spike_filter,
    'post_spike_filter': post_spike_filter,
    'pre_spike_sink': pre_spike_sink,
    'post_spike_sink': post_spike_sink
}
```

trip

implementation of triplet STDP algorithm if algorithm is 'triplet_stdp', which creates a filtering graph to generate Dynapse1Trace events with recorded neuron traces and update weights using LTP and LTD.

Type *triplet_stdp_details.TripletStdp*, optional

stdp_on

whether STDP is running or not.

Type bool

get_spikes_debug()

Get Spike events from the graph if spike_sink_debug is True.

Returns

- **pre_spikes** (*list[samna.dynapse1.Spike]*) – pre spikes.
- **post_spikes** (*list[samna.dynapse1.Spike]*) – post spikes.
- or **None** if **spike_sink_debug** is **False**.

start_stdp()

Start STDP learning algorithm so that weight update is turned on.

stop_stdp()

Stop STDP learning algorithm so that weight update is turned off.

stdp_utils

stdp_utils.bad_traces(*onpre_traces, onpost_traces, max_num=10, max_time_interval=300000.0*)

Checks if the received traces are abnormal. To handle the abnormal traces after restart of STDP. Bad traces should be dropped, not processed anymore. Traces are bad if

- 1) not synchronized: time difference between the traces is too large.
- 2) too many traces received which will take too long to process.

Parameters

- **onpre_traces** (*list[samna.dynapse1.Dynapse1Trace]*) – onpre traces which triggers LTD.
- **onpost_traces** (*list[samna.dynapse1.Dynapse1Trace]*) – onpost traces which triggers LTP.
- **max_num** (*int, optional*) – number of received traces at one `get_events()`. Defaults to 10.
- **max_time_interval** (*int, optional*) – time difference between the traces, in microsecond. Defaults to 3×10^5 .

Returns bool – True if the received traces are abnormal.

`stdp_utils.create_stdp_graph(model, spike_sink_debug=False)`

Create a filtering graph to track the neuron traces required by STDP-like learning algorithms. Graph:

source_node in model -> Dynapse1NeuronSelect filter_node -> Dynapse1NeuronTrace onpost and onpre trace filter_nodes -> onpost and onpre trace sink_nodes

Returned nodes:

- 'spike_filter': passes spikes of pre and post neurons.
- 'onpre_trace_filter': tracks and generates traces that are updated

at pre neurons' spiking times, which triggers LTD. - 'onpost_trace_filter': tracks and generates traces that are updated at post neurons' spiking times, which triggers LTP. - 'onpre_trace_sink': receives onpre trace events for LTD weight update. - 'onpost_trace_sink': receives onpost trace events for LTP weight update.

If spike_sink_debug is True, the following nodes will also be created:

- 'pre_spike_filter': passes pre neurons' spikes.
- 'post_spike_filter': passes post neurons' spikes.
- 'pre_spike_sink': receives pre neurons' spikes.
- 'post_spike_sink': receives post neurons' spikes.

Parameters

- **model** (*samna.dynapse1.Dynapse1Model*) – Dynapse1Model.
- **spike_sink_debug** (*bool, optional*) – whether to add spike filtering nodes for debugging. Defaults to False.

Returns

- **graph** (*samna.graph.EventFilterGraph*) – samna filtering graph.
- **nodes** (*dictionary{'str' – samna_nodes}*): See explanation above.

`stdp_utils.floatW2intW(float_w_plast, max_pre_count, rand_seed=None, unit=0.1)`

An example to do weight discretization. Converts a float weight matrix learned by STDP into an int matrix that can be applied to DYNAP-SE1 board. Here, 0.1 -> 1 connection

Parameters

- **float_w_plast** (*numpy.array*) – float weight matrix
- **max_pre_count** (*int*) – the maximum number of incoming synapses of a neuron.
- **rand_seed** (*int or None, optional*) – rand seed. Defaults to None.
- **unit** (*float, optional*) – unit value that will become 1 connection. Defaults to 0.1.

Returns `_type_ – _description_`

stdp_algorithms

class stdp_algorithms.triplet_stdp_details.TripletStdp(*param_file*)

Bases: object

Triplet STDP is implemented. Plastic connections between pre and post population are updated over time, including trace graph setup and algorithm implementation with retrieved traces. See details at <http://arxiv.org/abs/1608.08267>.

Other STDP-like learning algorithms can be implemented by

- **defining configuring a filtering graph to get required neuron traces** like *set_triplet_stdp_graph* for 1 pre trace and 2 post traces in triplet STDP.
- **implementing the weight update operations using received traces** as in *triplet_stdp_algorithm*.

Parameters *param_file* (*str*) – parameter file name that stores algorithm parameters.

method

how the neuron trace grows when a neuron fires. Can be “increase_by” or “increase_to” a constant value.

Type str

trace_max

maximum trace value since “increase_to” is chosen for triplet-STDP.

Type float

pre_tau

time constant of trace pre in microsecond. The unit is millisecond in JSON file for simplicity.

Type int

post1_tau

time constant of trace post1 in microsecond. The unit is millisecond in JSON file for simplicity.

Type int

post2_tau

time constant of trace post2 in microsecond. The unit is millisecond in JSON file for simplicity.

Type int

nuEEpre

presynaptic spike learning rate.

Type float

nuEEpost

postsynaptic spike learning rate.

Type float

wmaxEE

maximum weight of triplet-STDP.

Type float

expEEpre

presynaptic weight dependence.

Type float

expEEpost

postsynaptic weight dependence.

Type float

Algorithm:

On pre: neuron select node of pre -> pre timestamps update post_trace1: post_trace1 values at pre timestamps

$pre = 1.0$ $\Delta w = nuEEpre * post1 * w_{plast} * expEEpre$ $w_{plast} = clip(w_{plast} - \Delta w, 0, w_{maxEE})$

On post: neuron select node of post -> post timestamps update pre_trace: pre_trace values at post timestamps update post_trace2: post_trace2 values at post timestamps.

$post2_{before} = post2$ $\Delta w = nuEEpost * pre * post2_{before} * (w_{maxEE} - w_{plast}) * expEEpost$ $w_{plast} = clip(w_{plast} + \Delta w, 0, w_{maxEE})$ $post1 = 1.0$ $post2 = 1.0$

set_triplet_stdp_graph(*spike_filter_node, onpre_trace_node, onpost_trace_node, pre_neuron_ids, post_neuron_ids*)

Set up trace graph to calculate the required traces using related neurons. Here, pre and post2 traces are needed for LTP at post spiking times. Thus, the trigger neurons of these two traces are the post neurons as in onpost_trace_node. Similarly, post1 trace is required for LTD process at pre spiking times, and thus triggered by pre neurons in onpre_trace_node.

Parameters

- **spike_filter_node** – passes spikes of pre and post neurons.
- **onpre_trace_node** – tracks and generates traces that are updated at pre neurons' spiking times, which triggers LTD.
- **onpost_trace_node** – tracks and generates traces that are updated at post neurons' spiking times, which triggers LTP.
- **pre_neuron_ids** (*list[tuple(int, int, int)]*) – presynaptic neuron IDs (chip, core, neuron).
- **post_neuron_ids** (*list[tuple(int, int, int)]*) – postsynaptic neuron IDs (chip, core, neuron).

triplet_stdp_algorithm(*w_plast, onpre_traces, onpost_traces, pre_neuron_ids, post_neuron_ids*)

The implementation of triplet stdp algorithm given Dynapse1 traces retrieved from the trace graph. LTD when at pre spikes, LTP at post spikes using the pre, post1, post2 traces.

Parameters

- **w_plast** (*numpy.array*) – current plastic weight matrix, changing during learning.
- **onpre_traces** (*list[samna.dynapse1.Dynapse1Trace]*) – onpre traces which triggers LTD.
- **onpost_traces** (*list[samna.dynapse1.Dynapse1Trace]*) – onpost traces which triggers LTP.
- **pre_neuron_ids** (*list[tuple(int, int, int)]*) – presynaptic neuron IDs (chip, core, neuron).
- **post_neuron_ids** (*list[tuple(int, int, int)]*) – postsynaptic neuron IDs (chip, core, neuron).

Returns **w_plast** (*numpy.array*) – plastic weight matrix after this run of weight update.

1.4.5 Detailed netgen Implementation

The detailed implementation of NetworkGenerator is in *details/netgen_details.py*.

`details.netgen_details.check_and_write_post_cam(post, pre_core_id, pre_neuron_id, synapse_type, weight)`

Check and write post CAM.

`details.netgen_details.check_and_write_pre_sram(pre, post_chip_id, post_core_id)`

Check and write pre SRAM.

`details.netgen_details.convert_validated_network2dynapse1_configuration(network)`

Convert a validated “network” to a Dynapse1Configuration which can be applied using Dynapse1Model.

`details.netgen_details.find_neuron_in_dict(neuron, post_neuron_dict)`

Find neuron in the post_neuron_dict according neuron ID.

Parameters

- **neuron** (`netgen.Neuron`) – neuron
- **post_neuron_dict** (`dictionary`) – post_neuron_dict

Returns `netgen.Neuron` – neuron in the post_neuron_dict if neuron is in the dictionary, None otherwise.

`details.netgen_details.find_pre_in_post_incoming(pre_neuron, post_neuron_in_dict, synapse_type)`

Find pre in post incoming synapses.

`details.netgen_details.gen_all2all_lists(pre_neurons, post_neurons, p, rand_seed)`

Generate all-to-all lists.

`details.netgen_details.gen_neuron_string(neuron)`

Warning: `gen_neuron_string` is deprecated and will be removed in a future release, use `str(Neuron)` instead.

`details.netgen_details.gen_one2one_lists(pre_neurons, post_neurons)`

Generate one-to-one lists.

`details.netgen_details.get_usable_post_synapse_id(post_synapses, listen_core_id, listen_neuron_id, synapse_type, weight)`

check if the post neuron can receive a connection. .. attribute:: post_synapses

synapses of the post neuron, cam

`details.netgen_details.listen_core_id`

int, [0,4)

`details.netgen_details.listen_neuron_id`

int, [0,256)

`details.netgen_details.synapse_type`

`samna.dynapse1.Dynapse1SynType.NMDA, AMPA, GABA_B, GABA_A`

`details.netgen_details.weight`

number of cams needed to be written

`details.netgen_details.get_usable_pre_destination_id(pre_destinations, target_chip_id)`

check if the pre neuron can send a connection to the target chip. .. attribute:: pre_destinations

destinations of the pre neuron, sram

`details.netgen_details.target_chip_id`

int

`details.netgen_details.is_same_neuron_loc(neuron1, neuron2)`

Check if 2 neurons have the same physical ID. Consider a neuron as an individual neuron without any external connections (i.e. not in a Network)

Parameters *other* (`netgen.Neuron`) – a neuron.

Returns True if neuron IDs are the same, False otherwise.

`details.netgen_details.validate(network, max_num_cams=64)`

Validate if the network meets the DYNAP-SE1 hardware constraints.

Validation rules:

First check cams: check the number of incoming connections of each neuron (after reusing cam):

for each neuron, check its pre neurons:

if there's pre with same (core_id, neuron_id, synapse_type): calculate the number of cams (num_cams) needed by these pre neurons num_cams is the maximum weight of between a certain pre and this post other pre neurons will also get this max weight cause they are sharing the same cam send a warning to the user about 1) the weight sharing 2) different pres sharing same cam

if the number of incoming connections > 64: raise exception

Then check aliasing: for neurons in the same core, check their pre neurons. if two different post neurons have two different pres with same (core_id, neuron_id, synapse_type) but not in same chip: raise exception

No need to check srams

`details.netgen_details.weight_matrix2lists(weight_matrix, pre_group, post_group)`

convert w to pre_list and post_list.

`details.netgen_details.write_post_synapse(post_synapses, post_synapse_id, listen_core_id,
listen_neuron_id, synapse_type)`

Write post synapse.

`details.netgen_details.write_pre_destination(pre_destinations, pre_dest_id, virtual_core_id,
target_chip_id, sx, sy, dx, dy, core_mask, in_use)`

Write pre destination.

1.5 Debugging

1. “Failed to open USB device” error. Usually this can be solved if you unplug the USB cable and re-plug the DYNAP-SE1 board to your machine. If not, it’s probably due to the installation. You should update your udev rules by doing the following.

- Make a file in /etc/udev/rules.d/ called 60-inilabs.rules

```
cd /etc/udev/rules.d
cat 60-inilabs.rules
```

- Put the lines below into this file:

```
# All DVS/DAVIS/Dynap-se systems
SUBSYSTEM=="usb", ATTR{idVendor}=="152a", ATTR{idProduct}=="84[0-1]?", MODE="0666"
```

- Then, you run this command:

```
udevadm control --reload-rules
```

2. The GUI and *matplotlib.pyplot* may not work depending on your system.

- If you get the error below, please *open_dynapse1* without GUI. > samna.open_remote_node(visualizer_id, viz_name) RuntimeError: Store with ID: 3 timed out on content request
- If you get the error below, please avoid importing *matplotlib.pyplot*

together with Samna. > free(): invalid pointer [1] 18258 abort (core dumped)

1.6 APIs summary

dynapse1utils

netgen

params

stdp

stdp_utils

stdp_algorithms.triplet_std_details

details.netgen_details

1.6.1 dynapse1utils

Functions

<code>close_dynapse1(model[, gui_process])</code>	Close DYNAP-SE1 board with or without the GUI.
<code>create_neuron_select_graph(model, neuron_ids)</code>	Create a graph: source_node in model -> filter_node in graph -> sink_node to get events.
<code>free_port()</code>	Get a free port number using sockets.
<code>gen_destination_string(destination)</code>	Convert a Dynapse1Destination in the string format for printing: C<target_chip_id>c<core_mask><in_use>.
<code>gen_synapse_string(synapse)</code>	Convert a Dynapse1Synapse in the string format for printing: c<listen_core_id>n<listen_neuron_id><synapse_type>.
<code>get_Dynapse1_viz_converter()</code>	Helper function to reuse viz node for DYNAP-SE1 GUI.
<code>get_global_id(chip, core, neuron)</code>	Conveniently generate the corresponding global neuron ID given chip, core, neuron IDs.
<code>get_global_id_list(tuple_list)</code>	Conveniently generate a list of global neuron IDs given a list of [chip, core, neuron ID] tuples.
<code>get_neuron_from_config(config, chip, core, ...)</code>	Get a neuron by its global_neuron_id from a configuration.
<code>get_parameter_value(config, chip, core, name)</code>	Get the parameter (coarse, fine) value of a specific parameter in the DYNAP-SE1 configuration.
<code>get_parameters(config, chip, core)</code>	Get a list of Dynapse1Parameter (25 in total) of a specific core.
<code>get_selected_timestamps(spikes, neuron_ids)</code>	Given Dynapse1Spikes retrieved from NeuronSelect sinknode, filter out timestamps of selected neuron_ids.
<code>get_selected_traces(timed_traces, neuron_ids)</code>	Filter out timestamps and trace_values of selected neuron_ids.
<code>get_serial_number([device_idx, select_device])</code>	Get serial number of an opened DYNAP-SE1 board.
<code>get_time_wrap_events(model)</code>	DYNAP-SE1 sends out 2 types of events: spike and timeWrapEvent.
<code>get_trace_value(traces, timestamp)</code>	Get 1 single trace value from a list of Dynapse1Traces.
<code>open_device([sender_port, receiver_port, ...])</code>	Get unopened devices detected by samna.
<code>open_dynapse1([gui, select_device, ...])</code>	open DYNAP-SE1 board with or without GUI.
<code>open_gui(device[, visualizer_id])</code>	Open DYNAP-SE1 spiking GUI.
<code>open_visualizer(window_width, window_height, ...)</code>	start visualizer in a isolated process which is required on mac, instead of a sub process.
<code>print_dynapse1_spike(event)</code>	Print the Spike in the following format: "(timestamp, global_neuron_id)".
<code>print_neuron_destinations(neuron[, ...])</code>	Print the destinations of the neuron given the destination id list.
<code>print_neuron_synapses(neuron[, synapse_id_list])</code>	Print the synapses of the neuron given the synapse id list.
<code>save_parameters2json_file(config[, filename])</code>	Save the parameters of 16 cores into a JSON file.
<code>save_parameters2txt_file(config[, filename])</code>	Save the parameters of 16 cores into a txt file.
<code>set_fpga_spike_gen(fpga_spike_gen, ...[, ...])</code>	Author: Nicoletta Risi.
<code>set_parameters_in_json_file(model[, filename])</code>	Load the parameters in a JSON file to DYNAP-SE1 board.
<code>set_parameters_in_txt_file(model[, filename])</code>	Load the parameters in a txt file to DYNAP-SE1 board.

1.6.2 netgen

Functions

<i>add_synapses</i> (netgen, synapse)	Add a Synapses object into a network generator.
<i>add_wta_conns</i> (netgen, wta_conns)	Add WTA connections to create a WTA in the network generator.
<i>print_post_neuron_dict</i> (post_neuron_dict)	Print post_neuron_dict of the Network.
<i>remove_synapses</i> (netgen, synapse)	Remove a synapse group from a network generator.

Classes

<i>Network</i> ()	Network that checks and maintains the topology of a spiking neural network that meets DYNAP-SE1 hardware constraints.
<i>NetworkGenerator</i> ()	NetworkGenerator maintains a Network and a corresponding Dynapse1Configuration.
<i>Neuron</i> ([chip_id, core_id, neuron_id, ...])	Neuron object used to create network topology inside a network generator, Not the silicon neuron on DYNAP-SE1 hardware.
<i>NeuronGroup</i> ([chip_id, core_id, neuron_ids, ...])	A collection of netgen.Neurons for neural population operations.
<i>Synapses</i> (pre_group, post_group, synapse_type)	Stores the information of the connectivity from a pre NeuronGroup to a post NeuronGroup.
<i>WTA_connections</i> (exc_group, inh_group, ...[, ...])	Define EI IE EE connections of a WTA with an excitatory and an inhibitory population.

1.6.3 params

Functions

<i>gen_clean_param_group</i> ()	Generate a Dynapse1ParameterGroup of one core which should be able to silence the neurons.
<i>gen_dc_params</i> ()	Generate a Dynapse1ParameterGroup based on silent neurons, and turned DC current on.
<i>gen_param_group</i> ()	Generate a Dynapse1ParameterGroup of one core with some synapse weights turned on for examples.
<i>gen_stdp_params</i> ()	Generate a Dynapse1ParameterGroup for STDP example.
<i>set_params</i> (model[, dc, param_group])	Set 16 DYNAP-SE1 cores with the same Dynapse1ParameterGroup for examples.
<i>set_stdp_params</i> (model)	Set 16 DYNAP-SE1 cores with the same Dynapse1ParameterGroup for STDP example.

1.6.4 stdp

Classes

<i>Stdp</i> (model, net_gen, pre_neuron_ids, ...[, ...])	A class which implements STDP-like learning framework between a pre neuron population and a post neuron population with a computer in the loop.
--	---

1.6.5 stdp_utils

Functions

<i>bad_traces</i> (onpre_traces, onpost_traces[, ...])	Checks if the received traces are abnormal.
<i>create_stdp_graph</i> (model[, spike_sink_debug])	Create a filtering graph to track the neuron traces required by STDP-like learning algorithms. Graph: source_node in model -> Dynapse1NeuronSelect filter_node -> Dynapse1NeuronTrace onpost and onpre trace filter_nodes -> onpost and onpre trace sink_nodes.
<i>floatW2intW</i> (float_w_plast, max_pre_count[, ...])	An example to do weight discretization.

1.6.6 stdp_algorithms.triplet_stdp_details

Classes

<i>TripletStdp</i> (param_file)	Triplet STDP is implemented.
---------------------------------	------------------------------

1.6.7 details.netgen_details

Functions

<i>check_and_write_post_cam</i> (post, pre_core_id, ...)	Check and write post CAM.
<i>check_and_write_pre_sram</i> (pre, post_chip_id, ...)	Check and write pre SRAM.
<i>convert_validated_network2dynapse1_configuration</i> (network)	Convert (network) validated "network" to a Dynapse1Configuration which can be applied using Dynapse1Model.
<i>find_neuron_in_dict</i> (neuron, post_neuron_dict)	Find neuron in the post_neuron_dict according neuron ID.
<i>find_pre_in_post_incoming</i> (pre_neuron, ...)	Find pre in post incoming synapses.
<i>gen_all2all_lists</i> (pre_neurons, post_neurons, ...)	Generate all-to-all lists.
<i>gen_neuron_string</i> (neuron)	Warning: gen_neuron_string is deprecated and will be removed in a future release, use str(Neuron) instead.
<i>gen_one2one_lists</i> (pre_neurons, post_neurons)	Generate one-to-one lists.
<i>get_usable_post_synapse_id</i> (post_synapses, ...)	check if the post neuron can receive a connection.
<i>get_usable_pre_destination_id</i> (...)	check if the pre neuron can send a connection to the target chip.
<i>is_same_neuron_loc</i> (neuron1, neuron2)	Check if 2 neurons have the same physical ID.
<i>validate</i> (network[, max_num_cams])	Validate if the network meets the DYNAP-SE1 hardware constraints.
<i>weight_matrix2lists</i> (weight_matrix, ...)	convert w to pre_list and post_list.
<i>write_post_synapse</i> (post_synapses, ...)	Write post synapse.
<i>write_pre_destination</i> (pre_destinations, ...)	Write pre destination.

PYTHON MODULE INDEX

d

`details.netgen_details`, [32](#)

`dynapselutils`, [11](#)

n

`netgen`, [16](#)

p

`params`, [25](#)

s

`stdp`, [25](#)

`stdp_algorithms.triplet_stdp_details`, [30](#)

`stdp_utils`, [28](#)

INDEX

A

add_connection() (*netgen.Network* method), 17
 add_connection() (*netgen.NetworkGenerator* method), 18
 add_connections_from_list() (*netgen.Network* method), 17
 add_connections_from_list() (*netgen.NetworkGenerator* method), 18
 add_connections_from_type() (*netgen.Network* method), 17
 add_connections_from_type() (*netgen.NetworkGenerator* method), 18
 add_synapses() (*in module netgen*), 24
 add_wta_conns() (*in module netgen*), 24
 algorithm (*stdp.Stdp* attribute), 27

B

bad_traces() (*in module stdp_utils*), 28

C

check_and_write_post_cam() (*in module details.netgen_details*), 32
 check_and_write_pre_sram() (*in module details.netgen_details*), 32
 chip_id (*netgen.Neuron* attribute), 20
 chip_id (*netgen.NeuronGroup* attribute), 21
 clear_network() (*netgen.NetworkGenerator* method), 19
 close_dynapse1() (*in module dynapse1utils*), 11
 config (*netgen.NetworkGenerator* attribute), 18
 conn_type (*netgen.Synapses* attribute), 23
 convert_validated_network2dynapse1_configuration() (*in module details.netgen_details*), 32
 core_id (*netgen.Neuron* attribute), 20
 core_id (*netgen.NeuronGroup* attribute), 21
 create_neuron_select_graph() (*in module dynapse1utils*), 11
 create_stdp_graph() (*in module stdp_utils*), 29

D

details.netgen_details
 module, 32, 38

dynapse1utils
 module, 11, 34

E

ee (*netgen.WTA_connections* attribute), 23
 ei (*netgen.WTA_connections* attribute), 23
 expEEpost (*stdp_algorithms.triplet_stdp_details.TripletStdp* attribute), 30
 expEEpre (*stdp_algorithms.triplet_stdp_details.TripletStdp* attribute), 30

F

find_neuron_in_dict() (*in module details.netgen_details*), 32
 find_pre_in_post_incoming() (*in module details.netgen_details*), 32
 floatW2intW() (*in module stdp_utils*), 29
 free_port() (*in module dynapse1utils*), 12

G

gen_all2all_lists() (*in module details.netgen_details*), 32
 gen_clean_param_group() (*in module params*), 25
 gen_dc_params() (*in module params*), 25
 gen_destination_string() (*in module dynapse1utils*), 12
 gen_neuron_string() (*in module details.netgen_details*), 32
 gen_one2one_lists() (*in module details.netgen_details*), 32
 gen_param_group() (*in module params*), 25
 gen_stdp_params() (*in module params*), 25
 gen_synapse_string() (*in module dynapse1utils*), 12
 get_Dynapse1_viz_converter() (*in module dynapse1utils*), 12
 get_global_id() (*in module dynapse1utils*), 12
 get_global_id_list() (*in module dynapse1utils*), 12
 get_neuron_from_config() (*in module dynapse1utils*), 12
 get_parameter_value() (*in module dynapse1utils*), 13
 get_parameters() (*in module dynapse1utils*), 13

`get_selected_timestamps()` (in module `dynapse1utils`), 13
`get_selected_traces()` (in module `dynapse1utils`), 13
`get_serial_number()` (in module `dynapse1utils`), 13
`get_spikes_debug()` (`stdp.Stdp` method), 28
`get_time_wrap_events()` (in module `dynapse1utils`), 14
`get_trace_value()` (in module `dynapse1utils`), 14
`get_usable_post_synapse_id()` (in module `details.netgen_details`), 32
`get_usable_pre_destination_id()` (in module `details.netgen_details`), 32
`graph` (`stdp.Stdp` attribute), 27

I

`ie` (`netgen.WTA_connections` attribute), 23
`incoming_connections` (`netgen.Neuron` attribute), 21
`is_same_neuron_loc()` (in module `details.netgen_details`), 33
`is_spike_gen` (`netgen.Neuron` attribute), 21
`is_spike_gen` (`netgen.NeuronGroup` attribute), 21

L

`listen_core_id` (in module `details.netgen_details`), 32
`listen_neuron_id` (in module `details.netgen_details`), 32

M

`make_dynapse1_configuration()` (`netgen.NetworkGenerator` method), 19
`make_dynapse1_configuration_in_chip()` (`netgen.NetworkGenerator` method), 19
`make_dynapse1_configuration_in_core()` (`netgen.NetworkGenerator` method), 19
`max_time_interval` (`stdp.Stdp` attribute), 27
`max_trace_num` (`stdp.Stdp` attribute), 27
`method` (`stdp_algorithms.triplet_stdp_details.TripletStdp` attribute), 30
`model` (`stdp.Stdp` attribute), 26
`module`
 `details.netgen_details`, 32, 38
 `dynapse1utils`, 11, 34
 `netgen`, 16, 36
 `params`, 25, 36
 `stdp`, 25, 37
 `stdp_algorithms.triplet_stdp_details`, 30, 37
 `stdp_utils`, 28, 37
`mux_conn` (`netgen.Synapses` attribute), 23

N

`net_gen` (`stdp.Stdp` attribute), 26
`netgen`

 module, 16, 36
`Network` (class in `netgen`), 16
`network` (`netgen.NetworkGenerator` attribute), 18
`NetworkGenerator` (class in `netgen`), 18
`Neuron` (class in `netgen`), 20
`neuron_id` (`netgen.Neuron` attribute), 20
`neuron_ids` (`netgen.NeuronGroup` attribute), 21
`NeuronGroup` (class in `netgen`), 21
`neurons` (`netgen.NeuronGroup` property), 21
`new_thread` (`stdp.Stdp` attribute), 27
`nuEEpost` (`stdp_algorithms.triplet_stdp_details.TripletStdp` attribute), 30
`nuEEpre` (`stdp_algorithms.triplet_stdp_details.TripletStdp` attribute), 30

O

`open_device()` (in module `dynapse1utils`), 14
`open_dynapse1()` (in module `dynapse1utils`), 14
`open_gui()` (in module `dynapse1utils`), 15
`open_visualizer()` (in module `dynapse1utils`), 15

P

`p` (`netgen.Synapses` attribute), 23
`param_file` (`stdp.Stdp` attribute), 26
`params`
 module, 25, 36
`post1_tau` (`stdp_algorithms.triplet_stdp_details.TripletStdp` attribute), 30
`post2_tau` (`stdp_algorithms.triplet_stdp_details.TripletStdp` attribute), 30
`post_group` (`netgen.Synapses` attribute), 22
`post_list` (`netgen.Synapses` attribute), 22
`post_neuron_dict` (`netgen.Network` attribute), 16
`post_neuron_ids` (`stdp.Stdp` attribute), 26
`pre_group` (`netgen.Synapses` attribute), 22
`pre_list` (`netgen.Synapses` attribute), 22
`pre_neuron_ids` (`stdp.Stdp` attribute), 26
`pre_tau` (`stdp_algorithms.triplet_stdp_details.TripletStdp` attribute), 30
`print_dynapse1_spike()` (in module `dynapse1utils`), 15
`print_network()` (`netgen.NetworkGenerator` method), 20
`print_neuron_destinations()` (in module `dynapse1utils`), 15
`print_neuron_synapses()` (in module `dynapse1utils`), 15
`print_post_neuron_dict()` (in module `netgen`), 24

R

`rand_seed` (`netgen.Synapses` attribute), 23
`remove_bad_traces` (`stdp.Stdp` attribute), 27
`remove_connection()` (`netgen.Network` method), 17

`remove_connection()` (*netgen.NetworkGenerator* method), 20
`remove_connections_from_list()` (*netgen.NetworkGenerator* method), 18
`remove_connections_from_list()` (*netgen.NetworkGenerator* method), 20
`remove_synapses()` (in module *netgen*), 24

S

`save_parameters2json_file()` (in module *dynapse1utils*), 15
`save_parameters2txt_file()` (in module *dynapse1utils*), 15
`set_fpga_spike_gen()` (in module *dynapse1utils*), 15
`set_parameters_in_json_file()` (in module *dynapse1utils*), 16
`set_parameters_in_txt_file()` (in module *dynapse1utils*), 16
`set_params()` (in module *params*), 25
`set_stdp_params()` (in module *params*), 25
`set_triplet_stdp_graph()` (*stdp_algorithms.triplet_stdp_details.TripletStdp* method), 31
`spike_sink_debug` (*stdp.Stdp* attribute), 27
`start_stdp()` (*stdp.Stdp* method), 28
`stdp` module, 25, 37
`Stdp` (class in *stdp*), 25
`stdp_algorithms.triplet_stdp_details` module, 30, 37
`stdp_on` (*stdp.Stdp* attribute), 28
`stdp_utils` module, 28, 37
`stop_graph` (*stdp.Stdp* attribute), 27
`stop_stdp()` (*stdp.Stdp* method), 28
`synapse_type` (in module *details.netgen_details*), 32
`synapse_type` (*netgen.Synapses* attribute), 22
`Synapses` (class in *netgen*), 22

T

`target_chip_id` (in module *details.netgen_details*), 32
`trace_max` (*stdp_algorithms.triplet_stdp_details.TripletStdp* attribute), 30
`trip` (*stdp.Stdp* attribute), 28
`triplet_stdp_algorithm()` (*stdp_algorithms.triplet_stdp_details.TripletStdp* method), 31
`TripletStdp` (class in *stdp_algorithms.triplet_stdp_details*), 30
`tuple_neuron_ids` (*netgen.NeuronGroup* property), 21

V

`validate()` (in module *details.netgen_details*), 33

W

`w_plast` (*stdp.Stdp* attribute), 26
`weight` (in module *details.netgen_details*), 32
`weight_matrix` (*netgen.Synapses* attribute), 23
`weight_matrix2lists()` (in module *details.netgen_details*), 33
`wmaxEE` (*stdp_algorithms.triplet_stdp_details.TripletStdp* attribute), 30
`write_post_synapse()` (in module *details.netgen_details*), 33
`write_pre_destination()` (in module *details.netgen_details*), 33
`WTA_connections` (class in *netgen*), 23